
Rapport

IFT850 - Stage

Enseignants : Mikaël Fortin

Superviseur de stage : Pr Aref Meddeb

Étudiant : Mamadou Senghor

CIP : senm1912

Université de Sherbrooke

Département d'informatique

Hiver 2026



UNIVERSITÉ DE
SHERBROOKE

Date de remise : 1^{er} juin 2026

Table des matières

1	Introduction	4
1.1	Contexte général	4
1.2	Problématique	4
1.3	Objectifs du stage	4
1.4	Démarche et contributions	5
1.5	Organisation du rapport	5
2	Fondements théoriques de la cryptographie post-quantique	5
2.1	Définition et objectifs	5
2.2	Menaces quantiques sur la cryptographie classique	5
2.3	Principales familles de primitives post-quantiques	6
2.4	Signatures fondées sur les réseaux euclidiens	6
2.5	Signatures fondées sur les fonctions de hachage	6
2.6	Signatures fondées sur les isogénies	6
2.7	Processus de standardisation du NIST	6
3	Environnements matériels et logiciels	7
3.1	Présentation des plateformes matérielles	7
3.1.1	ESP32	7
3.1.2	Raspberry Pi	7
3.1.3	Particle Argon	7
3.2	Environnements de développement	7
3.2.1	Développement sur ESP32	7
3.2.2	Développement sur Raspberry Pi	7
3.2.3	Développement sur Particle Argon	8
3.3	Bibliothèques cryptographiques utilisées	8
3.4	Méthodologie de mesure des performances	8
4	Implémentation des mécanismes de signature post-quantique	9
4.1	Principes généraux d'implémentation	9
4.2	Choix des paramètres de sécurité	9
4.3	Implémentation de ML-DSA (Dilithium)	9
4.4	Implémentation de FALCON	9
4.5	Implémentation de SPHINCS+	10
4.6	Implémentation de SQIsign	10
4.7	Synthèse des difficultés et adaptations	10
4.8	Instrumentation pour la mesure énergétique	10
4.8.1	Mesure sur ESP32	11
4.8.2	Mesure sur Raspberry Pi	12
4.8.3	Mesure sur Particle Argon	13
4.8.4	Synchronisation des mesures	14
5	Campagnes de benchmarks et résultats expérimentaux	14
5.1	Vue d'ensemble des campagnes	14
5.2	Résultats du benchmark local : Raspberry Pi	14
5.2.1	Résultats temporels	15
5.2.2	Pic de mémoire dynamique	15
5.2.3	Décomposition mémoire	16
5.3	Résultats du benchmark local : ESP32	16
5.3.1	Résultats temporels	17

5.3.2	Mémoire totale mobilisée	17
5.3.3	Décomposition mémoire	18
5.3.4	Consommation énergétique	18
5.4	Résultats du benchmark local : Particle Argon	19
5.4.1	Temps d'exécution	19
5.4.2	Empreinte mémoire	20
5.4.3	Mémoire libre et contraintes système	20
5.4.4	Limitation de Falcon padded 1024	21
5.4.5	Consommation énergétique	21
5.4.6	Synthèse	22
5.5	Synthèse comparative globale	22
5.6	Benchmark inter-plateformes	22
6	Analyse et discussion des résultats	22
6.1	Analyse des performances temporelles	22
6.2	Analyse de l'empreinte mémoire	23
6.3	Analyse de la consommation énergétique	23
6.4	Impact des contraintes matérielles	24
6.5	Compromis et choix d'algorithmes	24
6.6	Limites de l'étude	24
6.7	Conclusion	25
7	Optimisations et améliorations	25
7.1	Optimisations au niveau de l'implémentation	25
7.2	Optimisations liées à la méthodologie expérimentale	26
7.3	Optimisations spécifiques aux plateformes	26
7.4	Optimisations énergétiques	26
7.5	Perspectives d'optimisation	27
7.6	Conclusion	27
8	Conclusion et perspectives	27

Résumé

L'essor de l'informatique quantique remet en cause la sécurité des mécanismes cryptographiques classiques et motive le développement de solutions résistantes aux attaques quantiques. Dans ce contexte, les systèmes embarqués, caractérisés par des ressources limitées en calcul, en mémoire et en énergie, constituent un environnement particulièrement contraint pour l'implémentation de mécanismes de signature post-quantique.

Ce rapport de stage présente une étude expérimentale comparative des performances de plusieurs schémas de signature post-quantique, notamment CRYSTALS-Dilithium (ML-DSA), FALCON et SPHINCS+, sur trois plateformes embarquées : ESP32, Raspberry Pi et Particle Argon. Les algorithmes sont évalués selon plusieurs critères, incluant le temps d'exécution des opérations cryptographiques (keypair, signature, vérification), l'empreinte mémoire et, lorsque possible, la consommation énergétique.

L'étude inclut également une analyse d'interopérabilité entre plateformes, dans laquelle une plateforme produit une signature qui est ensuite vérifiée par une autre, afin d'évaluer la compatibilité des implémentations dans un contexte distribué.

Les résultats obtenus à partir de campagnes de benchmarks systématiques mettent en évidence des compromis importants entre performance, consommation de ressources et contraintes matérielles propres à chaque plateforme. Cette analyse permet d'identifier les mécanismes les plus adaptés aux environnements embarqués et de formuler des recommandations pratiques pour l'intégration de signatures post-quantiques dans des systèmes contraints.

1 Introduction

1.1 Contexte général

La cryptographie joue un rôle central dans la sécurité des systèmes informatiques modernes, notamment pour garantir l'authenticité, l'intégrité et la confidentialité des données échangées. Les mécanismes de signature numérique sont largement utilisés dans de nombreux domaines, tels que les communications sécurisées, l'Internet des objets (IoT), les systèmes industriels et les infrastructures critiques.

Cependant, l'émergence de l'informatique quantique remet en cause la sécurité des schémas cryptographiques classiques actuellement déployés. En particulier, l'algorithme de Shor permet de résoudre efficacement certains problèmes mathématiques sur lesquels reposent les signatures numériques traditionnelles, comme RSA et les schémas basés sur les courbes elliptiques. Face à cette menace, la cryptographie post-quantique (Post-Quantum Cryptography, PQC) vise à concevoir des mécanismes cryptographiques résistants aux attaques menées par des ordinateurs quantiques.

Dans ce contexte, le National Institute of Standards and Technology (NIST) a lancé un vaste processus de standardisation afin de sélectionner des algorithmes post-quantiques destinés à remplacer les schémas actuels. Parmi ces algorithmes figurent plusieurs mécanismes de signature post-quantique reposant sur différentes hypothèses mathématiques, telles que les réseaux euclidiens, les fonctions de hachage et les isogénies de courbes elliptiques.

1.2 Problématique

Si les algorithmes de signature post-quantique offrent des garanties de sécurité face aux attaques quantiques, leur intégration dans des systèmes embarqués pose de nombreux défis. Ces plateformes sont généralement caractérisées par des ressources limitées en mémoire, en puissance de calcul et en énergie, ce qui complique l'implémentation d'algorithmes cryptographiques souvent plus coûteux que leurs équivalents classiques.

Les microcontrôleurs et systèmes embarqués utilisés dans l'IoT, l'industrie ou les systèmes autonomes doivent fréquemment assurer des opérations de signature et de vérification en temps réel, tout en respectant des contraintes strictes de consommation énergétique et de latence. Dans ce contexte, le choix d'un mécanisme de signature post-quantique ne peut se faire uniquement sur la base de critères de sécurité, mais doit également prendre en compte les performances et l'empreinte matérielle de l'algorithme.

De plus, bien que plusieurs études aient proposé des évaluations de performances des algorithmes PQC, celles-ci sont souvent limitées à une seule plateforme ou à des environnements peu contraints. Il existe donc un besoin réel de comparaisons homogènes et systématiques des mécanismes de signature post-quantique sur différentes plateformes embarquées.

1.3 Objectifs du stage

L'objectif général de ce stage est de comparer l'implémentation, les performances et les contraintes de plusieurs mécanismes de signature post-quantique sur des plateformes embarquées représentatives.

Plus précisément, les objectifs spécifiques sont les suivants :

- implémenter quatre mécanismes de signature post-quantique, à savoir CRYSTALS-Dilithium, FALCON, SPHINCS+ et SQIsign, sur trois plateformes embarquées : ESP32, Raspberry Pi et Particle Argon ;
- mesurer et analyser les performances de ces algorithmes selon différents critères, incluant le temps de signature, le temps de vérification, l'utilisation mémoire et, lorsque possible, la consommation énergétique ;

- comparer les résultats obtenus entre les différentes plateformes afin d'identifier l'impact des contraintes matérielles sur les performances cryptographiques ;
- proposer des recommandations pratiques pour le choix et l'intégration de mécanismes de signature post-quantique dans des environnements embarqués contraints.

1.4 Démarche et contributions

Afin d'atteindre ces objectifs, une démarche expérimentale structurée a été adoptée. Elle repose sur l'étude théorique des mécanismes de signature post-quantique, leur implémentation progressive sur les différentes plateformes embarquées, ainsi que la mise en place de campagnes de benchmarks systématiques.

Les contributions principales de ce travail incluent l'adaptation des algorithmes aux contraintes spécifiques de chaque plateforme, la conception d'un protocole de mesure homogène permettant des comparaisons fiables, ainsi que l'analyse approfondie des compromis entre sécurité, performances et consommation de ressources.

1.5 Organisation du rapport

Le reste de ce rapport est structuré comme suit. Le chapitre 2 présente les fondements théoriques de la cryptographie post-quantique ainsi qu'un état de l'art des principaux mécanismes de signature étudiés. Le chapitre 3 décrit les plateformes matérielles retenues et les environnements logiciels utilisés pour l'implémentation et l'évaluation. Le chapitre 4 détaille l'intégration et l'adaptation des algorithmes sur les systèmes embarqués. Le chapitre 5 est consacré aux campagnes de benchmarks réalisées et aux résultats expérimentaux obtenus, en évaluation locale et dans un scénario inter-plateformes. Le chapitre 6 propose une analyse et une discussion des résultats, en mettant en évidence les compromis entre performances, empreinte mémoire et consommation énergétique. Le chapitre 7 présente les optimisations explorées pour améliorer l'adéquation des algorithmes aux contraintes embarquées. Enfin, le chapitre 8 conclut le rapport et ouvre des perspectives pour des travaux futurs.

2 Fondements théoriques de la cryptographie post-quantique

2.1 Définition et objectifs

La cryptographie post-quantique (Post-Quantum Cryptography, PQC) désigne l'ensemble des mécanismes cryptographiques conçus pour résister aux attaques menées par des ordinateurs quantiques, tout en restant implémentables sur des architectures informatiques classiques [2].

Contrairement à la cryptographie quantique, qui exploite des phénomènes physiques issus de la mécanique quantique, la cryptographie post-quantique repose sur des problèmes mathématiques supposés difficiles à résoudre, même à l'aide des algorithmes quantiques connus à ce jour [2].

2.2 Menaces quantiques sur la cryptographie classique

Les schémas cryptographiques asymétriques classiques, tels que RSA, DSA et les mécanismes fondés sur les courbes elliptiques, reposent principalement sur la difficulté de la factorisation d'entiers et du calcul du logarithme discret.

L'algorithme quantique proposé par Shor permet de résoudre ces problèmes en temps polynomial sur un ordinateur quantique universel, compromettant ainsi la sécurité de ces schémas [1].

En revanche, les primitives symétriques et les fonctions de hachage sont principalement affectées par l'algorithme de Grover, qui offre un gain quadratique. Cette menace peut être partiellement atténuée par une augmentation appropriée de la taille des clés [2].

2.3 Principales familles de primitives post-quantiques

Les mécanismes de cryptographie post-quantique reposent sur plusieurs familles de problèmes mathématiques distincts, chacune présentant des propriétés spécifiques en termes de sécurité, de performances et d’empreinte mémoire [2].

Parmi ces familles figurent notamment les schémas fondés sur les réseaux euclidiens, les signatures basées sur les fonctions de hachage et les constructions utilisant les isogénies de courbes elliptiques supersingulières.

2.4 Signatures fondées sur les réseaux euclidiens

Les signatures fondées sur les réseaux constituent l’une des familles les plus étudiées et les plus prometteuses de la cryptographie post-quantique. Elles reposent sur la difficulté de problèmes tels que le *Short Integer Solution* (SIS) et le *Learning With Errors* (LWE), ainsi que leurs variantes structurées [12].

CRYSTALS-Dilithium et FALCON appartiennent à cette catégorie. Ces schémas offrent un bon compromis entre sécurité, performances et tailles des signatures, ce qui explique leur sélection par le NIST dans le cadre du processus de standardisation post-quantique [3].

2.5 Signatures fondées sur les fonctions de hachage

Les signatures basées sur les fonctions de hachage constituent une autre famille importante de mécanismes post-quantiques. Leur sécurité repose uniquement sur les propriétés cryptographiques des fonctions de hachage, telles que la résistance aux collisions et aux préimages [13].

SPHINCS+ est un schéma de signature sans état (*stateless*) appartenant à cette famille. Bien que ces mécanismes offrent des garanties de sécurité solides et bien comprises, ils se caractérisent généralement par des signatures de grande taille et des temps de calcul plus élevés, ce qui peut poser des défis dans les environnements embarqués contraints [7].

2.6 Signatures fondées sur les isogénies

Les signatures fondées sur les isogénies exploitent la difficulté du calcul d’isogénies entre courbes elliptiques supersingulières. Cette famille est plus récente et repose sur des hypothèses mathématiques différentes de celles des schémas fondés sur les réseaux ou les fonctions de hachage [14].

SQIsign est un exemple représentatif de cette approche. Il se distingue par des signatures très compactes, mais au prix de coûts de calcul élevés et d’implémentations complexes, ce qui soulève des questions quant à sa faisabilité sur des systèmes embarqués fortement contraints [9].

2.7 Processus de standardisation du NIST

Afin d’anticiper la transition vers des systèmes résistants aux attaques quantiques, le NIST a lancé en 2016 un processus de standardisation visant à sélectionner des algorithmes de cryptographie post-quantique [3].

À l’issue de plusieurs phases d’évaluation, CRYSTALS-Dilithium et SPHINCS+ ont été standardisés respectivement sous les noms ML-DSA (FIPS 204) et SLH-DSA (FIPS 205), tandis que FALCON a été retenu comme candidat pour une standardisation ultérieure. Les schémas fondés sur les isogénies, tels que SQIsign, n’ont pas été standardisés à ce stade, mais restent activement étudiés par la communauté scientifique.

3 Environnements matériels et logiciels

Ce chapitre présente les plateformes matérielles utilisées dans le cadre de ce stage, ainsi que les environnements logiciels et les outils de développement employés pour l'implémentation et l'évaluation des mécanismes de signature post-quantique. Ces choix ont été guidés par la volonté de couvrir un large spectre de systèmes embarqués, allant des microcontrôleurs contraints aux nano-ordinateurs disposant de ressources plus importantes.

3.1 Présentation des plateformes matérielles

3.1.1 ESP32

L'ESP32 est un microcontrôleur largement utilisé dans les applications IoT, intégrant des fonctionnalités Wi-Fi et Bluetooth. Il repose sur une architecture Xtensa dual-core cadencée jusqu'à 240 MHz et dispose de ressources limitées en mémoire, typiquement quelques centaines de kilo-octets de RAM et de mémoire Flash.

Ces caractéristiques font de l'ESP32 une plateforme représentative des environnements embarqués fortement contraints, où l'implémentation de mécanismes cryptographiques complexes, tels que les signatures post-quantiques, constitue un défi important en termes de performances et d'empreinte mémoire.

3.1.2 Raspberry Pi

Le Raspberry Pi est un nano-ordinateur monocarte exécutant un système d'exploitation Linux. Contrairement aux microcontrôleurs, il dispose de ressources matérielles beaucoup plus importantes, incluant un processeur multi-cœurs, plusieurs centaines de méga-octets de mémoire vive et un stockage persistant.

Dans ce travail, le Raspberry Pi est utilisé comme plateforme de référence afin d'établir des valeurs de performance dans un environnement peu contraint. Les résultats obtenus sur cette plateforme permettent de comparer les limitations introduites par les systèmes embarqués plus restreints, tels que l'ESP32 et le Particle Argon.

3.1.3 Particle Argon

Le Particle Argon est un microcontrôleur orienté IoT, basé sur le SoC Nordic nRF52840, intégrant des capacités de communication sans fil et une connectivité cloud via l'écosystème Particle. Il dispose d'environ 256 kB de RAM et 1 MB de mémoire Flash, le positionnant entre l'ESP32 et le Raspberry Pi en termes de ressources.

Cette plateforme est particulièrement intéressante pour l'étude de la faisabilité des mécanismes de signature post-quantique dans des contextes IoT industriels, où les contraintes matérielles sont fortes mais où des exigences élevées en matière de sécurité subsistent.

3.2 Environnements de développement

3.2.1 Développement sur ESP32

Le développement sur l'ESP32 est réalisé à l'aide du framework ESP-IDF, qui fournit un ensemble d'outils pour la programmation en langage C, la gestion des périphériques et le déploiement des applications sur le microcontrôleur. Cet environnement permet un contrôle fin des ressources matérielles et facilite l'instrumentation nécessaire aux mesures de performances.

3.2.2 Développement sur Raspberry Pi

Le Raspberry Pi exécute un système d'exploitation Linux, ce qui permet l'utilisation d'outils de développement standards tels que les compilateurs GNU et les bibliothèques cryptographiques

existantes. Cette plateforme offre un environnement flexible pour le prototypage, le débogage et le benchmarking des algorithmes de signature post-quantique.

3.2.3 Développement sur Particle Argon

Le développement sur le Particle Argon est effectué à l'aide de Particle Workbench, un environnement basé sur Visual Studio Code, en combinaison avec le Device OS et les outils cloud fournis par Particle. Bien que cet environnement simplifie le déploiement et la connectivité réseau, il impose également certaines contraintes liées à l'abstraction du matériel et à la gestion de la mémoire.

3.3 Bibliothèques cryptographiques utilisées

L'implémentation des mécanismes de signature post-quantique repose principalement sur des bibliothèques open source reconnues par la communauté scientifique.

Les implémentations de référence issues du projet PQClean ont été utilisées comme base pour les algorithmes CRYSTALS-Dilithium, FALCON, SPHINCS+ et SQIsign. Sur le Raspberry Pi, la bibliothèque `liboqs` a également été exploitée afin de faciliter l'intégration et le benchmarking des algorithmes standardisés par le NIST.

Des adaptations spécifiques ont été nécessaires pour les plateformes embarquées, notamment afin de réduire l'empreinte mémoire, de gérer l'absence de certaines fonctionnalités du système standard et d'assurer la compatibilité avec les chaînes de compilation propres à chaque environnement.

3.4 Méthodologie de mesure des performances

Afin de garantir des comparaisons fiables, reproductibles et équitables entre les différents mécanismes de signature post-quantique, une méthodologie de mesure homogène a été adoptée sur l'ensemble des plateformes testées.

Les performances ont été évaluées séparément pour chaque opération cryptographique (*key-pair*, *sign* et *verify*), afin d'identifier précisément le coût associé à chaque phase. Pour chaque configuration, les mesures ont été répétées sur un grand nombre d'itérations, précédées d'une phase de *warmup* permettant de stabiliser l'exécution et de limiter l'impact des effets transitoires, tels que les caches, les allocations dynamiques ou les initialisations internes.

Les temps d'exécution ont été mesurés à l'aide de chronomètres haute résolution basés sur des compteurs monotones, offrant une précision à l'échelle de la microseconde. Les métriques temporelles sont collectées individuellement à chaque itération, puis agrégées lors de l'analyse statistique (moyenne, médiane, percentiles).

L'utilisation mémoire a été évaluée en suivant l'évolution de la mémoire vive avant et après l'exécution des opérations cryptographiques, en tenant compte notamment de l'utilisation du *heap* et, lorsque applicable, de la mémoire résidente du processus. Cette approche permet d'estimer l'empreinte mémoire associée à chaque algorithme.

La consommation énergétique a été étudiée à l'aide d'un Power Profiler Kit II (PPK2). Des mesures fiables ont pu être obtenues sur ESP32.

Concernant le Raspberry Pi, bien que la chaîne d'instrumentation ait été validée (pilotage du PPK2, synchronisation par marqueurs logiciels et exécution distante), les mesures obtenues se sont révélées non exploitables en raison des contraintes d'alimentation du dispositif. Ce point est discuté en détail dans le chapitre 6.

L'ensemble des mesures temporelles, mémoires et énergétiques est enregistré sous forme de journaux structurés, facilitant leur post-traitement et l'analyse comparative entre les différentes variantes d'algorithmes et de plateformes.

4 Implémentation des mécanismes de signature post-quantique

Ce chapitre décrit l'implémentation des mécanismes de signature post-quantique étudiés dans le cadre de ce stage. Il présente les choix techniques réalisés, les adaptations nécessaires pour chaque plateforme embarquée ainsi que les principales difficultés rencontrées lors de l'intégration des algorithmes ML-DSA (anciennement CRYSTALS-Dilithium), FALCON, SPHINCS+ et SQIsign.

4.1 Principes généraux d'implémentation

L'implémentation des mécanismes de signature post-quantique repose sur une approche modulaire visant à faciliter la portabilité du code entre les différentes plateformes. Chaque algorithme est structuré autour de trois opérations fondamentales : la génération de paires de clés (*keypair*), la signature d'un message (*sign*) et la vérification de signature (*verify*).

Les implémentations de référence issues des projets *liboqs* et *PQClean* ont servi de base commune. Ces implémentations ont ensuite été adaptées afin de répondre aux contraintes spécifiques des plateformes embarquées, notamment en matière de gestion mémoire, de compatibilité avec les chaînes de compilation et de limitation des dépendances logicielles.

Une séparation claire a été maintenue entre le code cryptographique et les mécanismes d'instrumentation utilisés pour le benchmarking, afin de garantir la reproductibilité et la comparabilité des mesures.

4.2 Choix des paramètres de sécurité

Les paramètres de sécurité retenus pour chaque algorithme correspondent aux niveaux de sécurité définis par le NIST. Lorsque plusieurs variantes étaient disponibles, celles offrant un compromis raisonnable entre niveau de sécurité, complexité algorithmique et consommation de ressources ont été privilégiées.

Ce choix permet d'assurer une comparaison équitable entre les algorithmes, tout en restant représentatif de configurations susceptibles d'être déployées dans des systèmes embarqués réels.

4.3 Implémentation de ML-DSA (Dilithium)

ML-DSA est un schéma de signature fondé sur les réseaux euclidiens modulaires, reposant sur des opérations arithmétiques modulaires et des échantillonnages pseudo-aléatoires. Son implémentation s'appuie sur des primitives bien définies et largement étudiées, ce qui facilite son intégration sur différentes plateformes.

Sur le Raspberry Pi, l'algorithme bénéficie d'un environnement d'exécution riche, permettant d'obtenir des valeurs de référence fiables pour les temps d'exécution et l'utilisation mémoire. Sur les plateformes ESP32 et Particle Argon, des adaptations ont été nécessaires afin de limiter l'utilisation de la mémoire dynamique et de réduire l'empreinte mémoire globale.

Dans l'ensemble, ML-DSA présente une complexité d'implémentation modérée et des performances relativement stables, ce qui en fait un candidat pertinent pour une évaluation comparative sur systèmes embarqués.

4.4 Implémentation de FALCON

FALCON est également un schéma de signature fondé sur les réseaux, mais il se distingue par l'utilisation d'arithmétique en nombres réels et de structures mathématiques plus complexes. Cette particularité rend son implémentation plus délicate, en particulier sur les microcontrôleurs dépourvus d'unités de calcul en virgule flottante performantes.

Sur le Raspberry Pi, FALCON tire parti des capacités matérielles et logicielles d'un environnement Linux, ce qui permet une exécution relativement efficace. En revanche, sur les

plateformes ESP32 et Particle Argon, des difficultés ont été observées, notamment en lien avec la précision numérique, la consommation mémoire et les temps de calcul.

Ces contraintes peuvent limiter le déploiement de FALCON sur des systèmes embarqués fortement contraints, malgré ses avantages en termes de taille de signature.

4.5 Implémentation de SPHINCS+

SPHINCS+ est un schéma de signature post-quantique fondé exclusivement sur des fonctions de hachage et ne nécessitant pas de gestion d'état. Cette propriété simplifie son utilisation du point de vue fonctionnel, mais entraîne un coût computationnel élevé.

L'implémentation repose sur un grand nombre d'opérations de hachage, ce qui se traduit par des temps de signature et de vérification importants, en particulier sur les microcontrôleurs. De plus, la taille relativement élevée des signatures constitue une contrainte supplémentaire pour les plateformes disposant de ressources mémoire limitées.

Néanmoins, SPHINCS+ offre des garanties de sécurité solides et bien établies, ce qui en fait une solution pertinente lorsque la sécurité prime sur les contraintes de performance.

4.6 Implémentation de SQIsign

SQIsign est un schéma de signature fondé sur les isogénies de courbes elliptiques super-singulières. Il se distingue par des signatures très compactes, mais au prix d'une complexité algorithmique élevée et d'exigences importantes en ressources de calcul.

Sur le Raspberry Pi, l'implémentation de SQIsign permet d'évaluer la faisabilité et les performances de l'algorithme dans un environnement relativement peu contraint. En revanche, son intégration sur les plateformes ESP32 et Particle Argon s'est révélée plus complexe, principalement en raison des coûts de calcul et des exigences mémoire.

Ces observations montrent que, malgré des signatures compactes, SQIsign reste difficile à exploiter sur des systèmes embarqués fortement contraints sans optimisations substantielles.

4.7 Synthèse des difficultés et adaptations

L'implémentation des mécanismes de signature post-quantique sur des plateformes embarquées met en évidence plusieurs défis communs, notamment la gestion de la mémoire, les limitations de puissance de calcul et la compatibilité avec les environnements logiciels.

Des adaptations spécifiques ont été nécessaires pour chaque plateforme, telles que la réduction de l'utilisation de la mémoire dynamique, l'ajustement des paramètres d'exécution ou la limitation des fonctionnalités non essentielles. Ces adaptations constituent une étape clé pour rendre les algorithmes post-quantiques exploitables dans des contextes embarqués réalistes.

4.8 Instrumentation pour la mesure énergétique

Afin de mesurer la consommation énergétique des plateformes embarquées, nous avons utilisé le dispositif *Nordic Power Profiler Kit II (PPK2)* en mode ampèremètre.

Ce dispositif permet de mesurer le courant consommé par le système testé (DUT – *Device Under Test*) avec une fréquence d'échantillonnage élevée, tout en permettant une synchronisation précise avec les événements logiciels.

4.8.1 Mesure sur ESP32

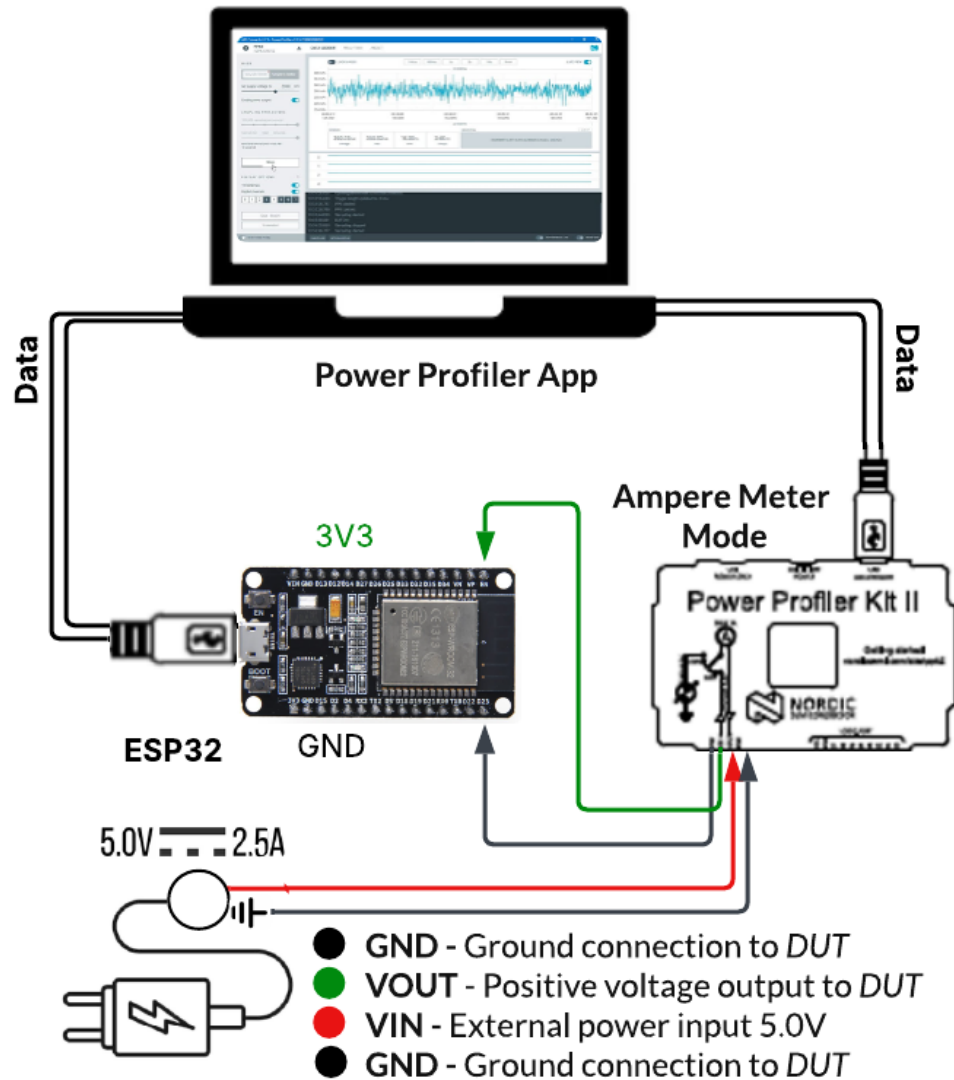


FIGURE 1 – Schéma de câblage pour la mesure énergétique sur ESP32 avec le PPK2 en mode ampèremètre.

Dans cette configuration, le PPK2 alimente directement l'ESP32 via sa sortie VOUT (3.3V). Le courant consommé est mesuré en série entre la source et le dispositif.

Ce montage permet une mesure fiable et directe de la consommation énergétique, car l'ensemble du courant traverse le PPK2.

4.8.2 Mesure sur Raspberry Pi

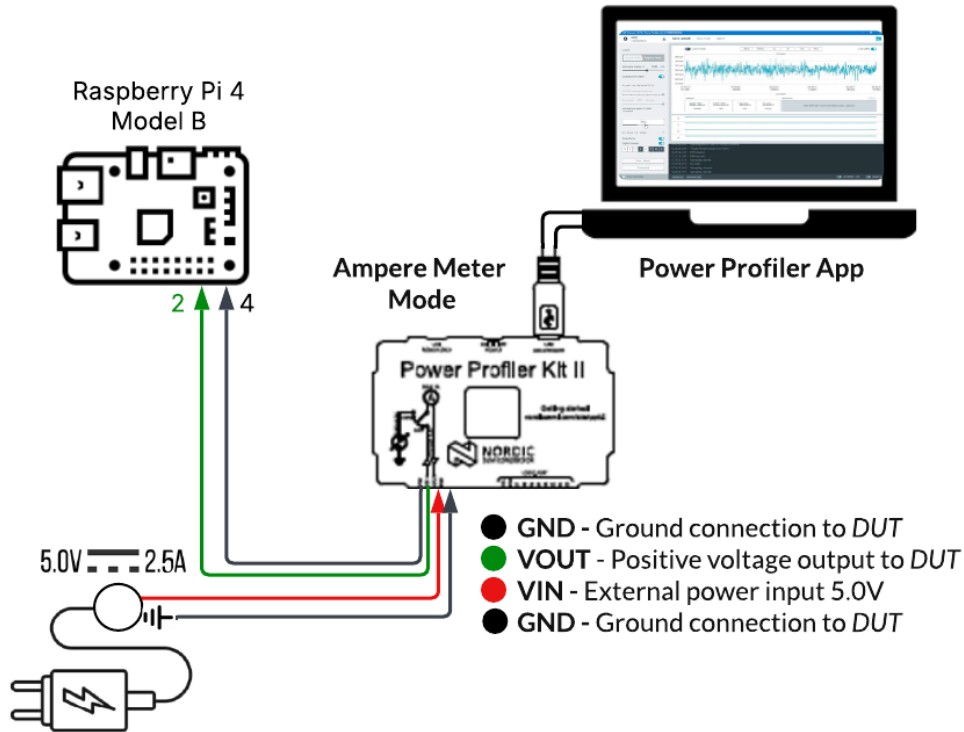


FIGURE 2 – Schéma de câblage pour la mesure énergétique sur Raspberry Pi avec le PPK2.

Contrairement à l'ESP32, le Raspberry Pi nécessite une alimentation externe (5V, courant élevé). Dans cette configuration, le PPK2 est utilisé en parallèle pour mesurer le courant.

Cependant, cette approche présente une limitation importante : le courant mesuré ne correspond pas à la consommation totale du système, car une partie du courant est fournie directement par l'alimentation externe.

4.8.3 Mesure sur Particle Argon

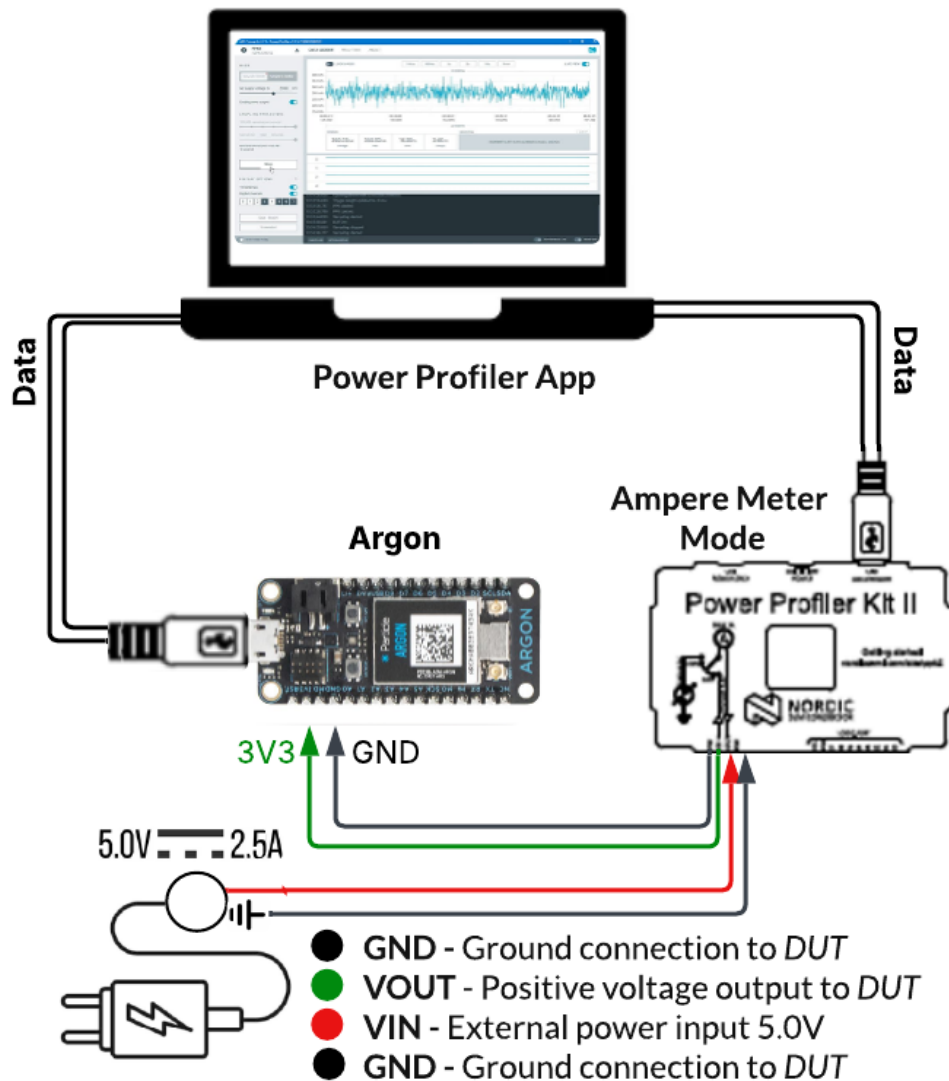


FIGURE 3 – Schéma de câblage pour la mesure énergétique sur Particle Argon avec le PPK2 en mode ampèremètre.

Dans cette configuration, le *Particle Argon* est alimenté directement par le PPK2 via la sortie VOUT (3.3V), similaire au montage utilisé pour l'ESP32.

Le courant consommé par le microcontrôleur traverse entièrement le PPK2, ce qui permet une mesure précise et fiable de la consommation énergétique globale du système.

Le PPK2 est configuré en mode *ampèremètre*, ce qui permet de capturer des variations fines du courant avec une fréquence d'échantillonnage élevée. Cette approche est particulièrement adaptée à l'analyse des opérations cryptographiques, dont les profils énergétiques peuvent varier rapidement.

Contrairement au Raspberry Pi, aucune alimentation externe supplémentaire n'est nécessaire, ce qui garantit que l'intégralité du courant mesuré correspond à celui consommé par le dispositif.

Ce montage permet ainsi une comparaison directe et cohérente des mesures énergétiques entre les différentes plateformes embarquées étudiées.

4.8.4 Synchronisation des mesures

Afin d'isoler précisément les phases de calcul, des marqueurs logiciels ont été utilisés :

- MEAS_START : début de la mesure ;
- MEAS_STOP : fin de la mesure.

Ces marqueurs permettent de synchroniser les traces énergétiques avec les opérations cryptographiques exécutées, garantissant une mesure précise et reproductible.

5 Campagnes de benchmarks et résultats expérimentaux

Ce chapitre présente les campagnes de benchmarks réalisées ainsi que les résultats expérimentaux obtenus sur les différentes plateformes étudiées. Les expériences ont été conçues selon une méthodologie homogène afin de garantir la comparabilité des mesures entre plateformes. Les différences observées proviennent principalement des caractéristiques matérielles et logicielles propres à chaque environnement d'exécution.

Les campagnes expérimentales sont organisées en deux volets complémentaires : le **benchmark local**, où les opérations cryptographiques sont exécutées directement sur une même plateforme, et le **benchmark inter-plateformes**, où une plateforme signe un message et une autre en effectue la vérification dans un scénario d'interopérabilité.

5.1 Vue d'ensemble des campagnes

Deux types d'expérimentations sont réalisés :

- **Benchmark local** : évaluation des performances intrinsèques des algorithmes sur chaque plateforme pour les opérations *keypair*, *sign*, *verify* et *whole*.
- **Benchmark inter-plateformes** : évaluation de l'interopérabilité entre plateformes, où une plateforme signe un message et une autre en effectue la vérification.

Les résultats sont présentés sous forme agrégée (médiane, moyenne) et analysés selon trois dimensions principales :

- le temps d'exécution ;
- l'empreinte mémoire ;
- la consommation énergétique (lorsqu'elle est disponible).

5.2 Résultats du benchmark local : Raspberry Pi

Cette section présente les résultats locaux obtenus sur la plateforme Raspberry Pi. Les Figures 4, 5 et 6 résument respectivement les temps d'exécution médians, le pic de mémoire dynamique et la décomposition mémoire pour l'opération *whole*.

5.2.1 Résultats temporels

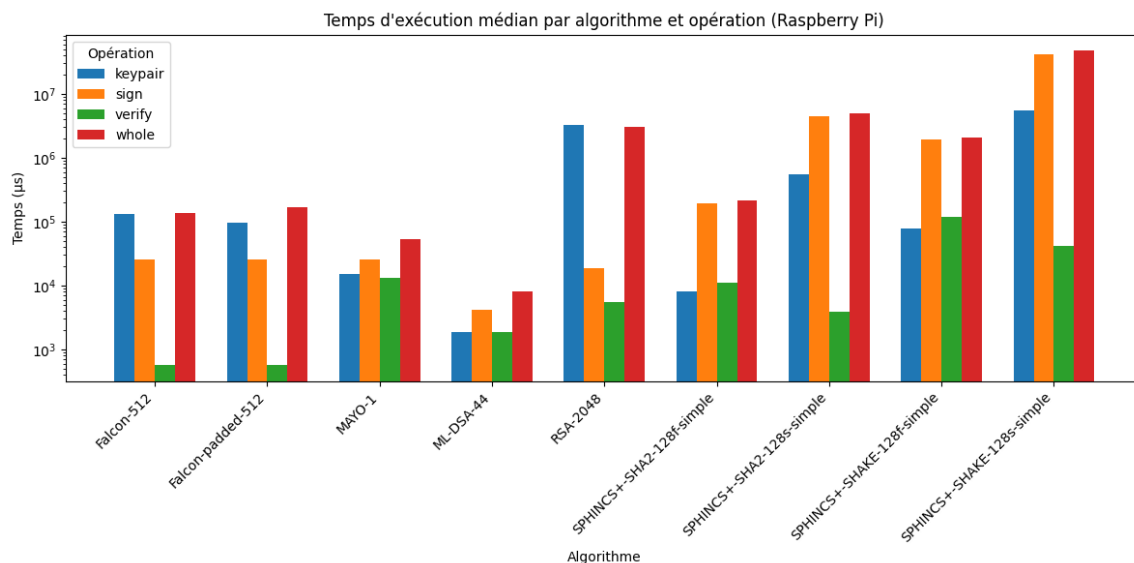


FIGURE 4 – Temps d'exécution médian par algorithme et opération sur Raspberry Pi (échelle logarithmique).

Les résultats montrent que les schémas fondés sur les réseaux, tels que *ML-DSA-44* et *FALCON-512*, présentent les meilleures performances globales. L'algorithme *MAYO-1* offre des performances intermédiaires.

RSA-2048 présente un comportement asymétrique classique : la génération de clés est coûteuse, tandis que la vérification est très rapide.

Les schémas *SPHINCS+* sont nettement plus lents, en particulier pour les opérations *sign* et *whole*, en raison de leur dépendance intensive aux fonctions de hachage.

5.2.2 Pic de mémoire dynamique

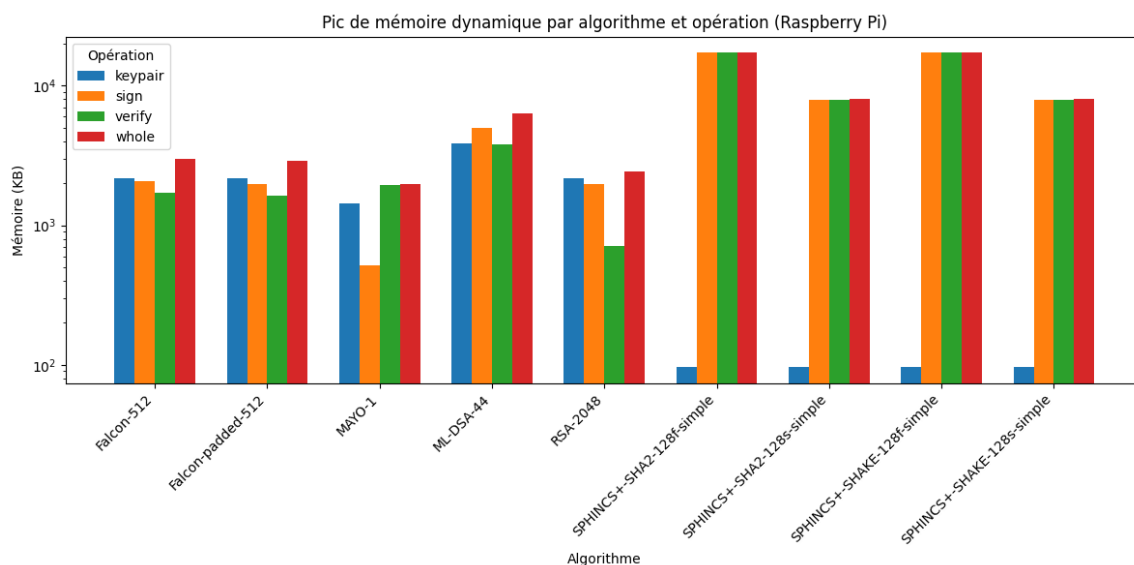


FIGURE 5 – Pic de mémoire dynamique par algorithme et opération sur Raspberry Pi.

Contrairement à la mémoire totale, fortement influencée par le système d'exploitation, le pic de mémoire dynamique met en évidence les différences réelles entre les algorithmes.

Les schémas SPHINCS+ présentent les consommations les plus élevées, atteignant plusieurs dizaines de mégaoctets pour certaines variantes, en particulier lors des opérations de signature et de vérification. Cette forte utilisation s'explique par la structure basée sur des arbres de hachage et le nombre important d'opérations internes.

À l'inverse, RSA-2048 se distingue par une empreinte mémoire dynamique très faible, notamment pour la génération de clés, ce qui reflète la maturité et l'optimisation des implémentations classiques.

Les algorithmes ML-DSA et Falcon occupent une position intermédiaire, avec une consommation modérée et relativement stable entre les différentes opérations.

Enfin, l'opération *whole* (cycle complet) correspond généralement au pire cas, combinant les coûts des différentes phases, ce qui explique les valeurs maximales observées.

5.2.3 Décomposition mémoire

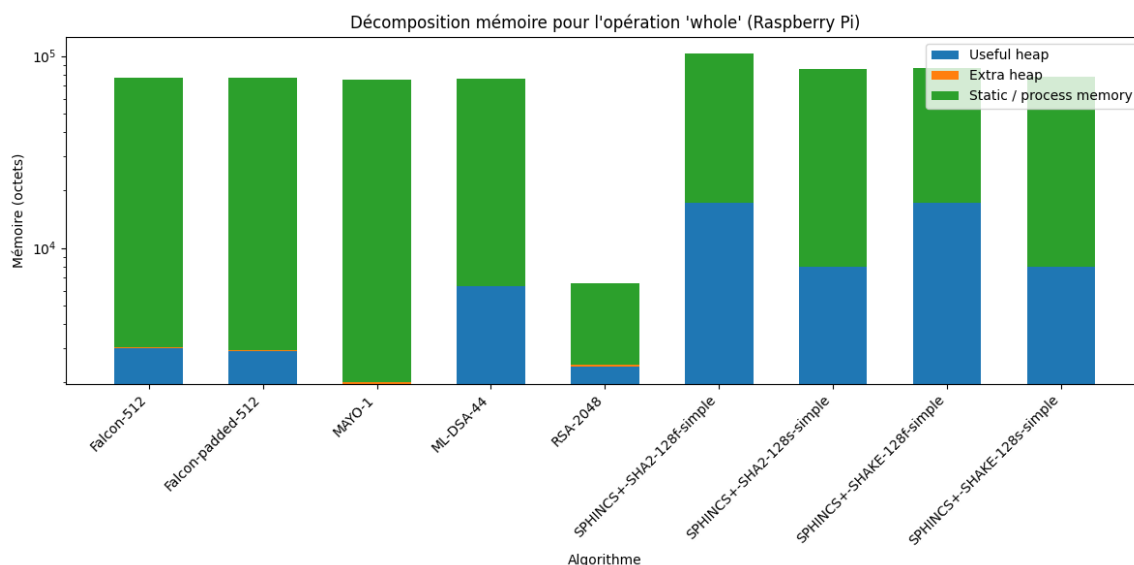


FIGURE 6 – Décomposition mémoire pour l'opération *whole* sur Raspberry Pi.

La mémoire est majoritairement dominée par la composante statique liée au processus. La mémoire dynamique reste modérée, et la fragmentation est négligeable.

Remarque énergétique La mesure de la consommation énergétique sur Raspberry Pi est en cours d'intégration et sera ajoutée dans une version ultérieure de ce chapitre.

5.3 Résultats du benchmark local : ESP32

Cette section présente les résultats obtenus sur ESP32, une plateforme fortement contrainte en ressources.

5.3.1 Résultats temporels

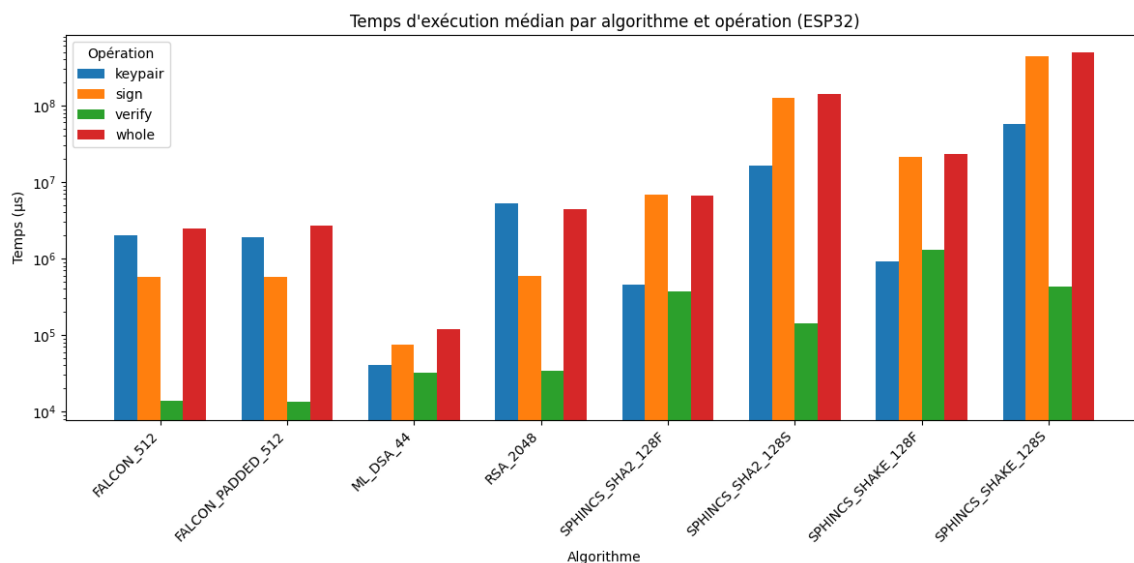


FIGURE 7 – Temps d'exécution médian par algorithme et opération sur ESP32 (échelle logarithmique).

Les schémas ML-DSA et FALCON sont les plus performants. Les schémas SPHINCS+ sont extrêmement coûteux, avec des temps plusieurs ordres de grandeur supérieurs.

RSA-2048 présente des performances intermédiaires.

5.3.2 Mémoire totale mobilisée

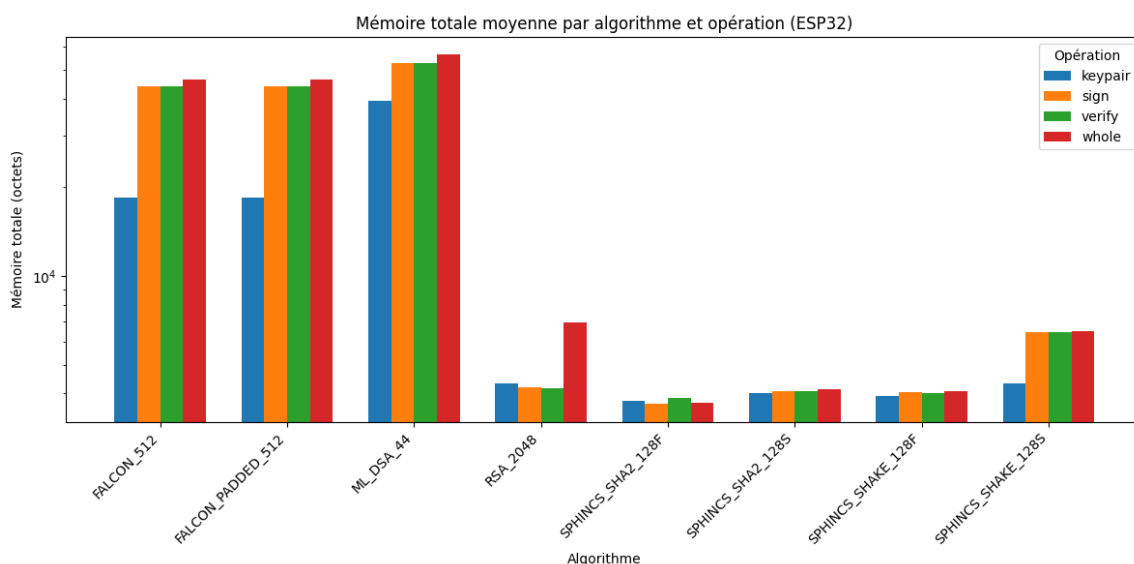


FIGURE 8 – Mémoire totale moyenne sur ESP32.

Les schémas ML-DSA et FALCON utilisent plus de mémoire que SPHINCS+, principalement en raison de leur utilisation de la pile.

5.3.3 Décomposition mémoire

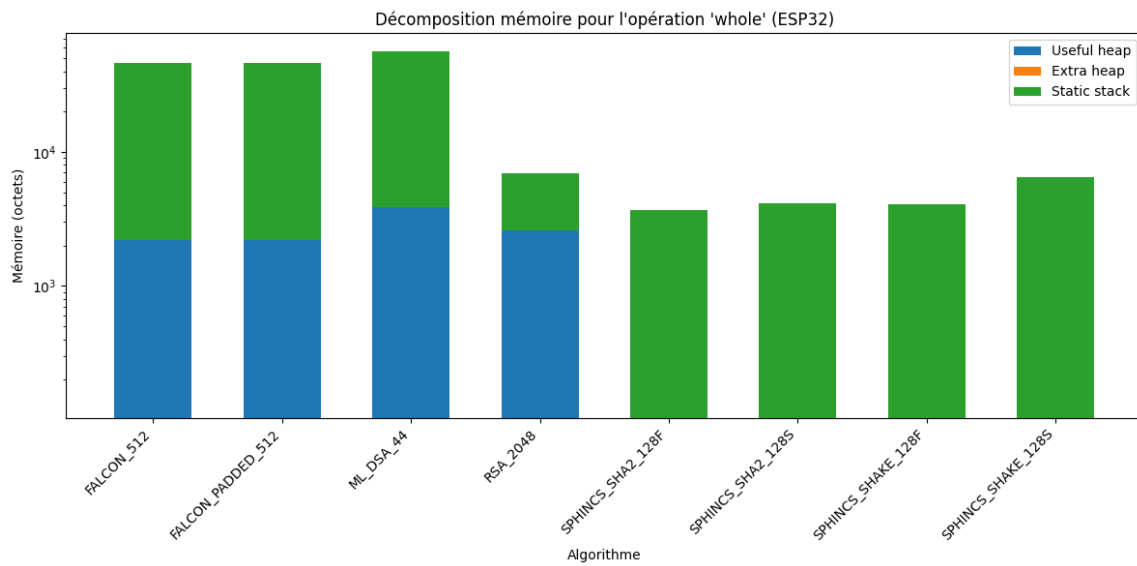


FIGURE 9 – Décomposition mémoire sur ESP32.

La mémoire statique (pile) est dominante. Le heap dynamique reste limité.

5.3.4 Consommation énergétique

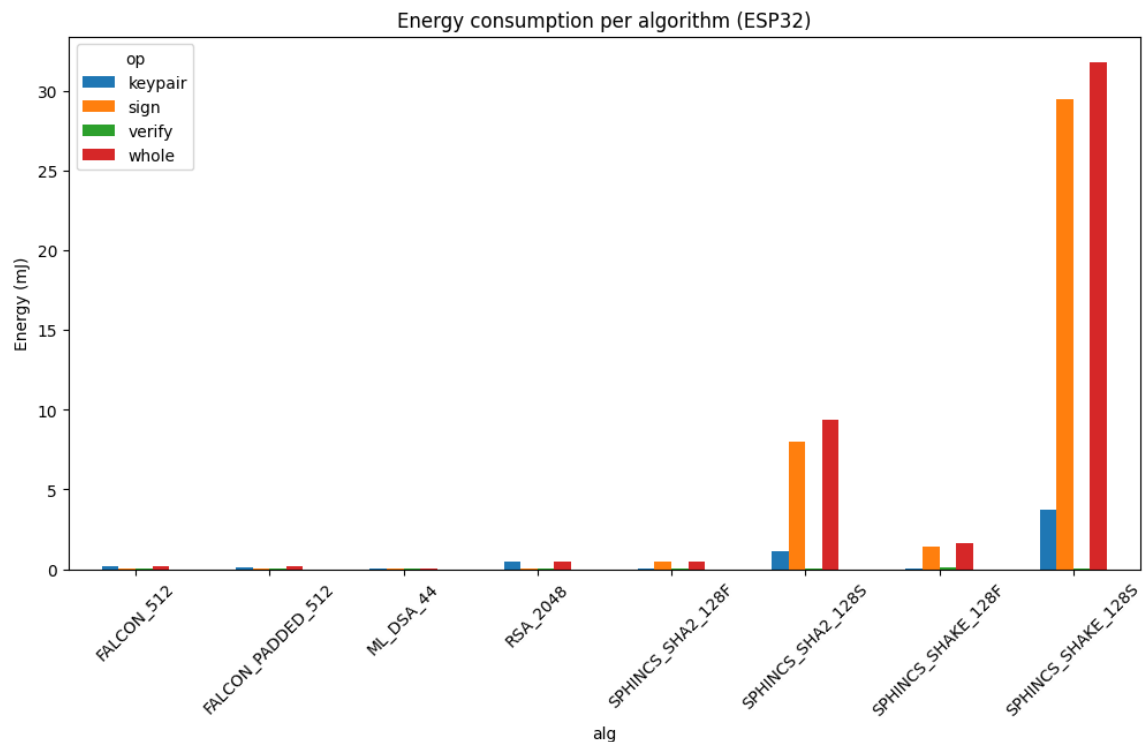


FIGURE 10 – Consommation énergétique sur ESP32 (mJ).

Les résultats montrent que ML-DSA et FALCON sont les plus efficaces énergétiquement. SPHINCS+ présente une consommation très élevée.

5.4 Résultats du benchmark local : Particle Argon

Cette section présente les résultats obtenus sur la plateforme Particle Argon, caractérisée par des ressources matérielles fortement contraintes. Les Figures 11, 12 et 13 illustrent respectivement les performances temporelles, l’empreinte mémoire et l’évolution de la mémoire libre.

5.4.1 Temps d’exécution

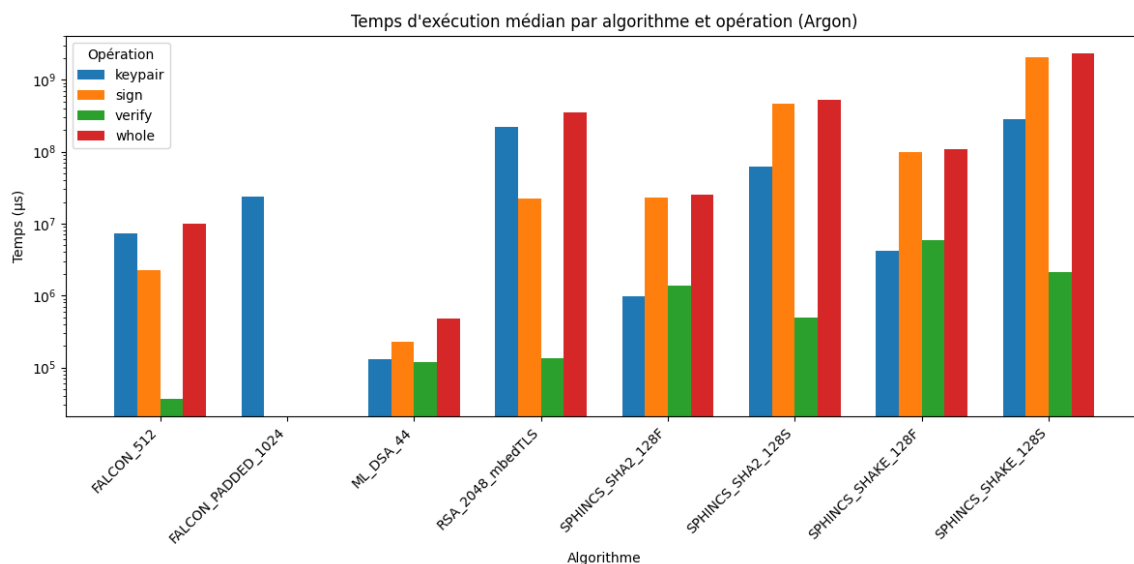


FIGURE 11 – Temps d’exécution médian par algorithme et opération sur Particle Argon (échelle logarithmique).

Les résultats montrent que les temps d’exécution sur Particle Argon sont significativement plus élevés que sur les autres plateformes étudiées (ESP32 et Raspberry Pi).

Cette différence s’explique par :

- une fréquence processeur plus faible ;
- des ressources matérielles limitées ;
- l’absence d’optimisations matérielles avancées.

Les schémas ML-DSA et FALCON restent les plus efficaces, bien que leurs performances soient fortement dégradées. À l’inverse, les algorithmes SPHINCS+ présentent des temps extrêmement élevés, en particulier pour *sign* et *whole*.

5.4.2 Empreinte mémoire

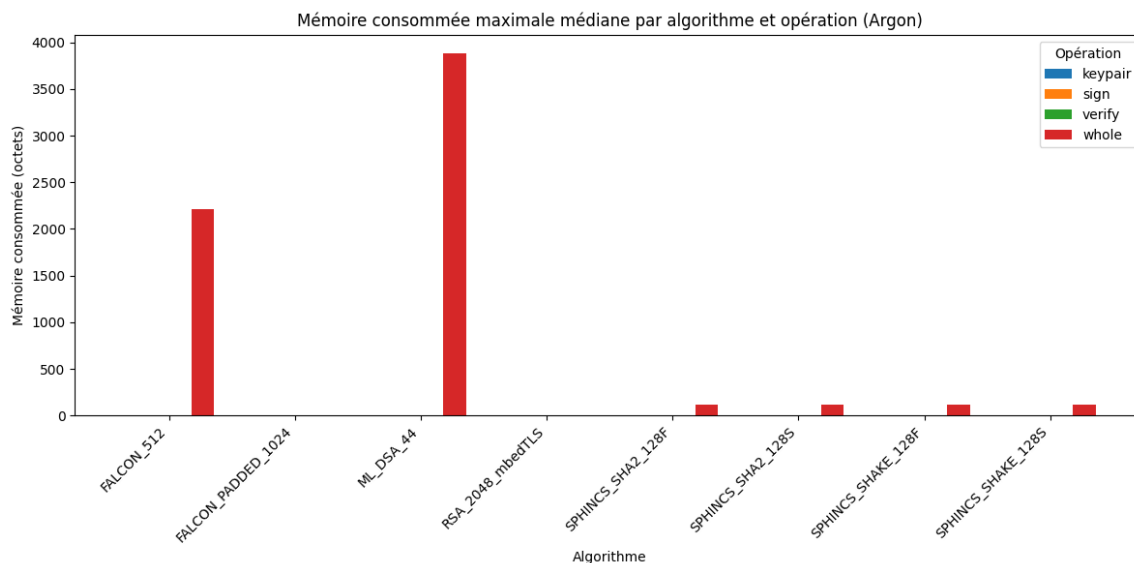


FIGURE 12 – Mémoire consommée maximale médiane par algorithme et opération sur Particle Argon.

L'analyse mémoire montre que la consommation reste globalement faible en termes absolus, ce qui est cohérent avec les contraintes de la plateforme.

La mémoire est majoritairement dominée par la pile (*stack*), tandis que l'utilisation du heap reste très limitée. Cela indique une implémentation adaptée aux environnements embarqués.

5.4.3 Mémoire libre et contraintes système

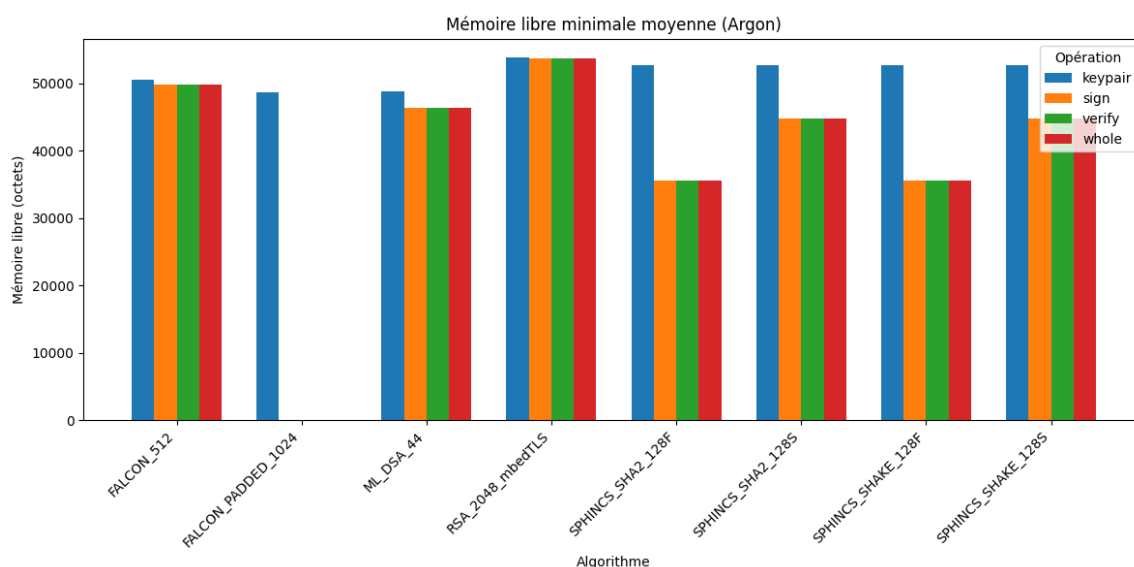


FIGURE 13 – Mémoire libre minimale moyenne par algorithme et opération sur Particle Argon.

L'analyse de la mémoire libre minimale montre une stabilité globale du système, avec des marges encore disponibles après exécution.

Cependant, certaines opérations (notamment SPHINCS+) entraînent une réduction notable

de la mémoire libre, ce qui peut constituer une contrainte pour des applications embarquées réelles.

5.4.4 Limitation de Falcon padded 1024

Une limitation importante a été observée concernant certaines variantes de FALCON sur la plateforme Particle Argon, en particulier *FALCON-1024* et *FALCON padded*.

Si l'opération *keypair* fonctionne correctement, les opérations *sign*, *verify* et *whole* échouent systématiquement dans l'état actuel de l'implémentation.

Des tests comparatifs ont montré que ces mêmes algorithmes s'exécutent correctement sur ESP32 et Raspberry Pi, ce qui suggère que le problème ne provient pas de l'implémentation logicielle, mais des contraintes matérielles spécifiques à la plateforme Argon.

Cette limitation est très probablement liée à :

- une consommation élevée de mémoire (notamment la pile) pour ces variantes ;
- des besoins accrus en calcul flottant et en structures intermédiaires ;
- une capacité mémoire plus restreinte du Particle Argon (256 kB de RAM).

Ces observations indiquent que certaines variantes de FALCON, en particulier les configurations de sécurité élevées, peuvent dépasser les capacités des microcontrôleurs fortement contraints sans optimisations supplémentaires.

5.4.5 Consommation énergétique

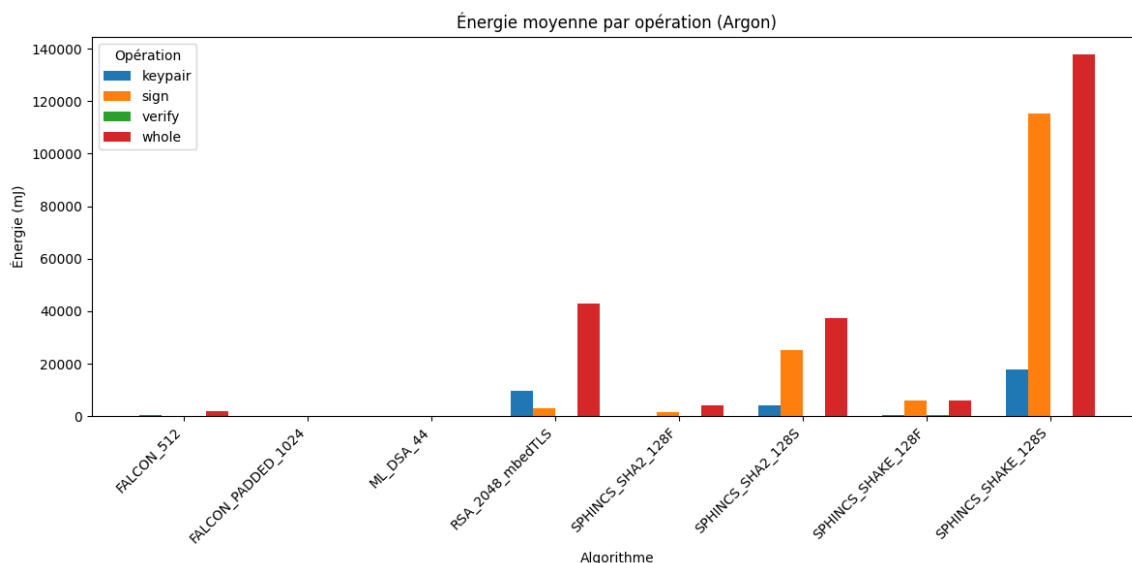


FIGURE 14 – Énergie moyenne consommée par algorithme et opération sur Particle Argon.

La consommation énergétique a été mesurée à l'aide du Power Profiler Kit II (PPK2) en distinguant une phase de référence (*idle*) et une phase d'exécution (*run*), permettant d'isoler la contribution des opérations cryptographiques.

Les résultats montrent une forte variabilité entre les algorithmes, en cohérence avec les mesures temporelles.

Les schémas ML-DSA présentent une très faible consommation énergétique, avec des valeurs de l'ordre de quelques dizaines de millijoules par opération, ce qui en fait les plus efficaces sur cette plateforme.

FALCON présente une consommation modérée, supérieure à ML-DSA mais restant compatible avec les contraintes embarquées, notamment pour les opérations de vérification.

À l'inverse, les schémas SPHINCS+ présentent des coûts énergétiques très élevés, en particulier pour les opérations de signature, certaines variantes dépassant largement plusieurs dizaines de joules. Ces résultats confirment leur coût computationnel important.

La consommation énergétique apparaît fortement corrélée au temps d'exécution, les opérations les plus longues étant également les plus énergivores.

5.4.6 Synthèse

Les résultats obtenus sur Particle Argon mettent en évidence les limites des plateformes microcontrôleurs pour l'exécution de signatures post-quantiques complexes.

Les schémas ML-DSA et FALCON offrent le meilleur compromis global entre performance, consommation mémoire et efficacité énergétique. À l'inverse, les schémas SPHINCS+ présentent des coûts prohibitifs en temps et en énergie, limitant leur utilisation dans des environnements embarqués contraints.

Ces observations soulignent l'importance du choix algorithmique dans un contexte embarqué, où les ressources disponibles sont fortement limitées.

5.5 Synthèse comparative globale

Les résultats montrent que :

- ML-DSA et FALCON offrent le meilleur compromis global.
- SPHINCS+ est très coûteux en temps et en énergie.
- RSA présente un comportement asymétrique.
- Les contraintes matérielles amplifient fortement les différences.

5.6 Benchmark inter-plateformes

Le scénario consiste à signer sur une plateforme et vérifier sur une autre.

Les métriques incluent :

- taux de succès
- temps
- mémoire
- énergie

Cette partie sera complétée ultérieurement.

6 Analyse et discussion des résultats

Ce chapitre propose une analyse approfondie des résultats expérimentaux obtenus lors des campagnes de benchmarks. L'objectif est d'identifier les tendances majeures, de comparer les algorithmes entre eux et de mettre en évidence les compromis entre performance, empreinte mémoire et consommation énergétique dans un contexte embarqué.

6.1 Analyse des performances temporelles

Les résultats mettent en évidence des différences significatives entre les familles d'algorithmes, avec des écarts particulièrement marqués sur les plateformes embarquées.

Algorithmes basés sur les réseaux (ML-DSA, FALCON) Les schémas ML-DSA et FALCON présentent les meilleures performances globales sur l'ensemble des plateformes. Ils offrent des temps d'exécution faibles et relativement stables, y compris sur des dispositifs contraints comme l'ESP32 et le Particle Argon.

FALCON est particulièrement performant pour l'opération de vérification, tandis que ML-DSA présente un comportement plus équilibré entre les différentes opérations. Ces résultats confirment l'efficacité des schémas basés sur les réseaux pour des environnements contraints.

Algorithmes à base de hachage (SPHINCS+) Les schémas SPHINCS+ sont de loin les plus coûteux en termes de temps d'exécution. Les opérations de signature et de traitement complet (*whole*) présentent des latences très élevées.

Ce comportement s'explique par l'utilisation intensive de fonctions de hachage et de structures arborescentes, impliquant un grand nombre d'opérations élémentaires. Sur les microcontrôleurs, ces coûts sont fortement amplifiés, rendant ces schémas difficilement exploitables en pratique.

RSA-2048 RSA-2048 présente un comportement asymétrique classique :

- la génération de clés est coûteuse ;
- la signature est modérée ;
- la vérification est très rapide.

Ce profil contraste avec les schémas post-quantiques et met en évidence les différences fondamentales entre cryptographie classique et post-quantique.

6.2 Analyse de l'empreinte mémoire

L'analyse mémoire révèle plusieurs tendances importantes.

Dominance de la mémoire statique Sur les plateformes embarquées (ESP32, Argon), la mémoire utilisée est majoritairement liée à la pile (stack). Cela indique que les implémentations reposent fortement sur des allocations statiques ou semi-statiques, adaptées aux contraintes des systèmes embarqués.

Faible utilisation du heap La mémoire dynamique (heap) reste globalement faible pour la majorité des algorithmes. Cette caractéristique est particulièrement visible sur ESP32, où les variations de heap sont limitées.

Différences entre familles d'algorithmes Les schémas ML-DSA et FALCON consomment davantage de mémoire que SPHINCS+, en raison de la manipulation de structures mathématiques plus complexes (vecteurs, matrices, polynômes).

À l'inverse, SPHINCS+, bien que coûteux en temps, présente une empreinte mémoire relativement modérée, ce qui constitue un avantage dans des environnements fortement contraints en mémoire.

6.3 Analyse de la consommation énergétique

Les mesures énergétiques réalisées sur les plateformes embarquées (ESP32 et Particle Argon) mettent en évidence des différences significatives entre les algorithmes.

Algorithmes efficaces ML-DSA et FALCON présentent les meilleures performances énergétiques. Leur faible temps d'exécution se traduit directement par une consommation énergétique réduite, ce qui en fait des candidats particulièrement adaptés aux systèmes embarqués à faible consommation.

Algorithmes coûteux Les schémas SPHINCS+ présentent une consommation énergétique très élevée, principalement due à leur temps d'exécution extrêmement long. Sur Argon, certaines variantes atteignent des coûts énergétiques de plusieurs dizaines, voire centaines de joules par opération, ce qui limite fortement leur utilisation dans des contextes contraints en énergie.

Corrélation temps-énergie Une corrélation forte est observée entre le temps d'exécution et la consommation énergétique. Les algorithmes les plus lents sont également les plus énergivores, ce qui est cohérent dans un contexte où la puissance instantanée reste relativement stable.

6.4 Impact des contraintes matérielles

Les différences entre plateformes mettent en évidence l'impact des contraintes matérielles sur les performances.

Raspberry Pi vs microcontrôleurs La Raspberry Pi offre des performances largement supérieures grâce à :

- une fréquence CPU plus élevée ;
- une mémoire plus importante ;
- un environnement système complet.

À l'inverse, les microcontrôleurs (ESP32, Argon) amplifient fortement les coûts computationnels et énergétiques, en particulier pour les algorithmes complexes.

Effet amplificateur Les écarts entre algorithmes sont beaucoup plus marqués sur les plateformes embarquées. Les schémas coûteux comme SPHINCS+ deviennent difficilement utilisables, tandis que les schémas plus efficaces comme ML-DSA et FALCON restent exploitables.

6.5 Compromis et choix d'algorithmes

Les résultats mettent en évidence des compromis clairs entre les différentes métriques :

- **Performance vs sécurité** : SPHINCS+ offre des garanties fortes mais à un coût très élevé ;
- **Temps vs mémoire** : ML-DSA et FALCON sont rapides mais peuvent exercer une pression mémoire plus importante ;
- **Énergie vs efficacité** : les algorithmes rapides sont également les plus économes en énergie ;
- **Interopérabilité vs pérennité** : RSA reste performant mais n'est pas résistant aux attaques quantiques.

6.6 Limites de l'étude

Malgré les résultats obtenus, certaines limitations expérimentales doivent être soulignées :

- les mesures énergétiques ne sont pas encore disponibles pour toutes les plateformes ;
- certaines implémentations (ex. Falcon padded sur Argon) ne sont pas entièrement supportées ;
- les résultats dépendent des choix d'implémentation et d'optimisation ;
- les mesures sont réalisées dans un environnement contrôlé (sans charge système significative).

Mesures énergétiques sur Raspberry Pi Les essais de mesure énergétique sur Raspberry Pi ont permis de valider la chaîne logicielle d'instrumentation, incluant le pilotage du PPK2, la synchronisation via des marqueurs logiciels et l'exécution distante des benchmarks.

Cependant, les valeurs de courant mesurées se sont révélées non physiques. Cette anomalie s'explique par la nécessité d'alimenter le Raspberry Pi à l'aide d'une source externe, le PPK2 n'étant pas capable de fournir un courant suffisant pour assurer son fonctionnement.

Dans cette configuration, le courant mesuré par le PPK2 ne correspond pas à la consommation réelle du système, mais uniquement à une fraction du courant total, rendant les résultats non exploitables.

En conséquence, les mesures énergétiques sur Raspberry Pi ont été écartées de l'analyse. Ce point constitue une limite expérimentale importante et suggère l'utilisation d'un dispositif de mesure adapté aux systèmes à plus forte consommation dans des travaux futurs.

6.7 Conclusion

Cette étude met en évidence que le choix d'un algorithme de signature post-quantique dépend fortement du contexte d'utilisation.

Les schémas ML-DSA et FALCON apparaissent comme les meilleurs candidats pour les systèmes embarqués, offrant un bon compromis entre performance, mémoire et énergie.

À l'inverse, les schémas SPHINCS+, bien que robustes, présentent des coûts trop élevés pour de nombreuses applications embarquées.

Ces résultats soulignent l'importance d'une approche de *crypto-agilité*, permettant d'adapter dynamiquement les mécanismes cryptographiques aux contraintes du système cible.

7 Optimisations et améliorations

Ce chapitre présente les optimisations explorées afin d'améliorer les performances des mécanismes de signature post-quantiques dans un contexte embarqué. L'objectif est de réduire le coût en temps d'exécution, en mémoire et en énergie, tout en conservant la validité fonctionnelle des implémentations.

7.1 Optimisations au niveau de l'implémentation

Réduction des allocations dynamiques Une première optimisation consiste à limiter les allocations dynamiques (heap), en privilégiant des buffers statiques ou réutilisables.

Sur les plateformes embarquées, cette approche permet :

- de réduire la fragmentation mémoire ;
- de limiter les appels à l'allocateur ;
- d'améliorer la stabilité des mesures.

Dans l'implémentation ESP32 et Argon, les buffers nécessaires aux opérations cryptographiques sont alloués une seule fois puis réutilisés, ce qui réduit significativement le coût mémoire.

Pré-génération des clés Pour les opérations *sign* et *verify*, la génération de clés est effectuée en dehors de la fenêtre de mesure.

Cette optimisation permet :

- d'isoler le coût réel des opérations étudiées ;
- de réduire la variabilité des résultats ;
- d'éviter des biais dans les mesures énergétiques.

Réduction des entrées/sorties Les écritures sur sortie standard (logs, JSON) sont mises en tampon (buffering) afin de limiter l'impact des opérations d'E/S sur les performances.

Sur Raspberry Pi, un buffer de grande taille est utilisé pour écrire les résultats JSONL, réduisant ainsi les perturbations liées au système de fichiers.

7.2 Optimisations liées à la méthodologie expérimentale

Warmup et stabilisation Une phase de *warmup* est utilisée avant les mesures afin de stabiliser :

- les caches ;
- l'état mémoire ;
- les effets liés à l'initialisation des bibliothèques.

Cette approche améliore la reproductibilité des résultats.

Mesures haute précision L'utilisation d'horloges monotoniques à haute résolution permet d'obtenir des mesures précises en microsecondes, essentielles pour comparer des algorithmes rapides.

Isolation des mesures énergétiques L'utilisation de marqueurs logiciels (`MEAS_START`, `MEAS_STOP`) permet de synchroniser précisément les mesures énergétiques avec les opérations exécutées.

Cela garantit que seules les portions pertinentes du code sont prises en compte dans les calculs d'énergie.

7.3 Optimisations spécifiques aux plateformes

ESP32 Sur ESP32, plusieurs optimisations ont été appliquées :

- utilisation de délais (`vTaskDelay`) pour stabiliser l'exécution ;
- suivi du *stack high-water mark* pour analyser la mémoire ;
- limitation des allocations dynamiques ;
- instrumentation fine des opérations.

Ces optimisations permettent d'obtenir des mesures fiables malgré les contraintes du microcontrôleur.

Raspberry Pi Sur Raspberry Pi, les optimisations suivantes ont été mises en place :

- exécution sur un cœur CPU dédié (`taskset`) ;
- réduction des perturbations liées au système ;
- buffering des sorties JSONL ;
- utilisation de bibliothèques optimisées (liboqs, OpenSSL).

Particle Argon Sur Particle Argon, les optimisations ont principalement porté sur :

- l'intégration des implémentations PQClean ;
- l'adaptation du générateur aléatoire ;
- la gestion mémoire dans un environnement fortement contraint ;
- la simplification des chemins d'exécution.

Cependant, certaines limitations subsistent, notamment pour les implémentations incomplètes (ex. Falcon padded).

7.4 Optimisations énergétiques

Réduction du temps d'exécution La principale stratégie d'optimisation énergétique consiste à réduire le temps d'exécution, en raison de la corrélation directe entre temps et énergie.

Isolation des phases de mesure L'utilisation d'un mode *idle-match* permet de comparer :

- une phase d'exécution réelle ;
- une phase sans calcul mais avec un comportement similaire.

Cela permet d'isoler la consommation énergétique spécifique aux opérations cryptographiques.

Choix des algorithmes Le choix d’algorithmes plus efficaces (ML-DSA, FALCON) constitue en soi une optimisation énergétique importante, notamment pour les systèmes embarqués.

7.5 Perspectives d’optimisation

Plusieurs pistes d’amélioration peuvent être envisagées :

- optimisation bas niveau (SIMD, assembleur) pour certaines primitives ;
- réduction de la taille des clés et signatures ;
- utilisation d’accélérateurs matériels cryptographiques ;
- optimisation des implémentations PQClean pour les architectures embarquées ;
- amélioration des mécanismes de gestion mémoire.

7.6 Conclusion

Les optimisations mises en œuvre ont permis d’améliorer significativement la qualité des mesures et les performances globales des implémentations.

Cependant, des marges d’amélioration subsistent, notamment pour les algorithmes les plus coûteux et les plateformes les plus contraintes. Ces résultats confirment l’importance d’adapter les implémentations cryptographiques aux caractéristiques spécifiques des systèmes embarqués.

8 Conclusion et perspectives

Ce stage avait pour objectif d’évaluer expérimentalement les performances de mécanismes de signature post-quantique dans un contexte embarqué, caractérisé par des contraintes fortes en termes de calcul, de mémoire et d’énergie.

Dans ce cadre, plusieurs algorithmes de signature post-quantique, notamment CRYSTALS-Dilithium (ML-DSA), FALCON et SPHINCS+, ont été intégrés et évalués sur différentes plateformes représentatives : ESP32, Raspberry Pi et Particle Argon. Une méthodologie expérimentale rigoureuse a été mise en place, incluant des mesures temporelles précises, une instrumentation mémoire fine et une analyse énergétique basée sur le Power Profiler Kit II.

Les résultats obtenus mettent en évidence des différences significatives entre les familles d’algorithmes. Les schémas basés sur les réseaux, tels que ML-DSA et FALCON, offrent les meilleures performances globales et apparaissent comme les plus adaptés aux systèmes embarqués. À l’inverse, les schémas à base de hachage comme SPHINCS+ présentent des coûts computationnels et énergétiques très élevés, limitant leur utilisation dans des environnements contraints.

L’étude a également permis de valider la faisabilité de scénarios d’interopérabilité entre plateformes hétérogènes, démontrant la compatibilité fonctionnelle des implémentations et leur intégration dans des systèmes distribués.

Cependant, certaines limitations ont été identifiées. Les mesures énergétiques n’ont pas pu être exploitées sur Raspberry Pi en raison de contraintes liées à l’alimentation du dispositif. De plus, certaines implémentations restent partielles sur les microcontrôleurs, notamment pour certaines variantes d’algorithmes.

Plusieurs perspectives de travail peuvent être envisagées. Il serait pertinent d’améliorer les mesures énergétiques sur des plateformes à plus forte consommation en utilisant des instruments adaptés. Des optimisations supplémentaires pourraient également être explorées, notamment au niveau bas niveau (SIMD, assembleur) ou via l’utilisation d’accélérateurs matériels. Enfin, l’extension de l’étude à d’autres algorithmes post-quantiques et à des scénarios applicatifs réels permettrait de renforcer la portée des résultats obtenus.

En conclusion, ce travail met en évidence l’importance d’une approche de *crypto-agilité*, permettant d’adapter dynamiquement les mécanismes cryptographiques aux contraintes du système cible. Les résultats obtenus constituent une base solide pour l’intégration de solutions post-quantiques dans les systèmes embarqués de demain.

Références

- [1] Shor, P. W. (1994). Algorithms for quantum computation : Discrete logarithms and factoring. *Proceedings of the 35th Annual Symposium on Foundations of Computer Science (FOCS)*, 124–134.
- [2] Bernstein, D. J., Buchmann, J., & Dahmen, E. (2009). *Post-Quantum Cryptography*. Springer.
- [3] National Institute of Standards and Technology (NIST). (2016–2025). Post-Quantum Cryptography Standardization Project. Available at : <https://csrc.nist.gov/projects/post-quantum-cryptography>
- [4] Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schwabe, P., Seiler, G., & Stehlé, D. (2018). CRYSTALS-Dilithium : A lattice-based digital signature scheme. *IACR Cryptology ePrint Archive*, Report 2017/633.
- [5] National Institute of Standards and Technology (NIST). (2024). FIPS 204 : Module-Lattice-Based Digital Signature Standard (ML-DSA).
- [6] Fouque, P.-A., Hoffstein, J., Kirchner, P., Lyubashevsky, V., Pornin, T., Prest, T., Ricosset, T., Seiler, G., Whyte, W., & Zhang, Z. (2019). Falcon : Fast-Fourier lattice-based compact signatures over NTRU lattices. *IACR Cryptology ePrint Archive*, Report 2018/1098.
- [7] Bernstein, D. J., Dobraunig, C., Eichlseder, M., Fluhrer, S., Gazdag, S., Kampanakis, P., Kölbl, S., Lange, T., Lauridsen, M., Mendel, F., & Niederhagen, R. (2019). SPHINCS+ : Stateless hash-based signatures. *IACR Cryptology ePrint Archive*, Report 2019/1086.
- [8] National Institute of Standards and Technology (NIST). (2024). FIPS 205 : Stateless Hash-Based Digital Signature Standard (SLH-DSA).
- [9] De Feo, L., Kohel, D., Leroux, A., Petit, C., & Wesolowski, B. (2020). SQIsign : Compact post-quantum signatures from isogenies. *IACR Cryptology ePrint Archive*, Report 2020/1240.
- [10] Koç, Ç. K., Acar, T., & Kaliski, B. S. (1996). Analyzing and comparing Montgomery multiplication algorithms. *IEEE Micro*, 16(3), 26–33.
- [11] Jain, R. (1991). *The Art of Computer Systems Performance Analysis*. John Wiley & Sons.
- [12] Micciancio, D., & Regev, O. (2009). Lattice-based cryptography. In Bernstein, D. J., Buchmann, J., & Dahmen, E. (Eds.), *Post-Quantum Cryptography*. Springer.
- [13] Buchmann, J., Dahmen, E., & Schneider, M. (2013). Post-quantum cryptography : State of the art. *IEEE Security & Privacy*, 11(4), 24–34.
- [14] De Feo, L. (2018). Mathematics of isogeny-based cryptography. *arXiv preprint arXiv :1711.04062*.